

Technical Documentation:

Autonomous Web Agent

Framework Updates

This document provides an in-depth technical analysis of the recent architectural upgrades to the Autonomous Web Agent Framework. It details the underlying mechanics, algorithms, and integration strategies for each major deliverable.

1. Hybrid DOM/Vision Fallback Architecture

To improve resilience against heavily dynamic sites and anti-bot modals, a Hybrid extraction and execution architecture was implemented.

Technical Implementation

- **Execution Wrap (`browser/actions.py`):** The `BrowserExecutor.execute()` method handles Playwright exceptions (e.g., `TimeoutError`, element unreachability). If a standard DOM-based click fails, it returns `False` rather than crashing the loop.
- **Agent Fallback Logic (`agents/reasoning_agent.py`):**
 - The agent checks for `action_success == False` or `action.fallback_to_vision == True`.
 - Upon failure, the agent dynamically creates a `/screenshots` directory.
 - It utilizes `page.screenshot()` to capture the active viewport state (saving as `vision_fallback_[timestamp].png`).

- The system is prepared to pass this screenshot to a Multi-Modal LLM to receive explicit X/Y coordinates to bypass obscured DOM elements (e.g., cookie banners).
-

2. Token-Optimized Viewport Extraction

To prevent rate-limit crashes (`HTTP 413`) on massive DOM structures (like Booking.com) and to conform to strict LLM Token-Per-Minute (TPM) limits, the extraction pipeline was overhauled.

Technical Implementation

- **Viewport Mathematical Filtering** (`browser/dom_extractor.py`):
 - Injected an `isInViewport(e1)` JavaScript function.
 - It calculates spatial intersection: `rect.top < window.innerHeight && rect.bottom > 0`.
 - Elements outside the active render view are aggressively dropped, mimicking a human's limited field of view and dropping element extraction counts from ~2500+ to ~50.
 - **Micro-Formatting** (`context/context_builder.py`):
 - The context builder algorithm was tightened to remove JSON overhead. Elements are serialized into dense strings (e.g., `pw-id-4 [INPUT] 'Search' p1:'Where to?'`).
 - A hard-cap safety threshold truncates the context string if it exceeds 15,000 characters (~3,500 tokens).
-

3. Semantic Memory & ChromaDB Integration

The agent needed long-term state retention and the ability to detect cyclical failure loops.

Technical Implementation

- **Vector Database (`memory/history.py`):**
 - Integrated `chromadb` to act as an ephemeral, in-memory vector store for the session.
 - Extracted page contexts and user intents are vectorized.
 - **Loop Detection Algorithm:**
 - The `MemoryState` tracks a sequential history of `AgentAction` objects.
 - `detect_loop()` performs a sliding-window analysis on the last 3 actions. If the exact same action and target (`pw-id`) are repeated 3 times, a loop is flagged.
 - Upon detection, the reasoning agent dynamically overrides the LLM's decision with a `scroll` action to force a change in the viewport state.
-

4. Advanced DOM Traversal & Obscurement Detection

Standard Playwright locators often fail when websites utilize Web Components or heavy Z-index layering.

Technical Implementation

- **Shadow DOM Piercing (`dom_extractor.py`):**
- The JS extraction script recursively traverses `node.shadowRoot` . It flattens encapsulated Web Components into the primary extraction array, ensuring custom elements are not invisible to the LLM.
- **Z-Index Modal Detection:**

- Implemented an `isObscured(el)` function utilizing `document.elementFromPoint(centerX, centerY)`.
 - It mathematically checks if the element at the target's center coordinates matches the target itself. If a modal (like a cookie popup) intercepts the point, the underlying element is excluded from the context, forcing the LLM to interact with the modal first.
-

5. Extended Physical Action Support

The `BrowserExecutor` was expanded to support complex interactions beyond simple clicks and text entry.

Technical Implementation

- **Pydantic Model Updates (`models/action_models.py`):**
- Expanded `Literal` action types to include `hover`, `drag_to`, `press_key`, and `select`.
- Added `x`, `y`, and `fallback_to_vision` fields for coordinate-based physical execution.
- **Playwright Bindings (`browser/actions.py`):**
- **Hover:** Binds to `locator.hover()`.
- **Drag & Drop:** Binds to `locator.drag_to()` utilizing a secondary target parser.
- **Coordinate Clicks:** If `x` and `y` are provided (via Vision fallback), it executes raw `mouse.click(x, y)` to bypass DOM locators entirely.
- **Multi-Match Recovery:** If a selector yields multiple elements (e.g., `.nth(0)` fails), it defaults to `.first` to ensure execution doesn't halt on duplicate IDs.